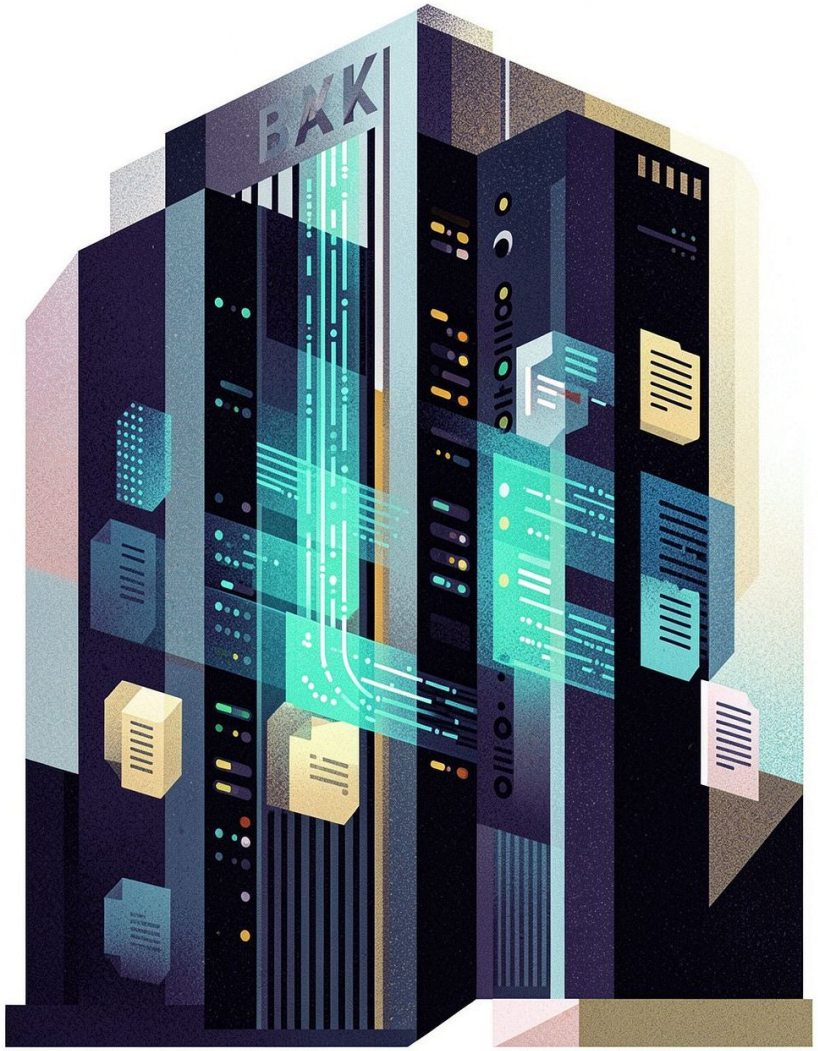


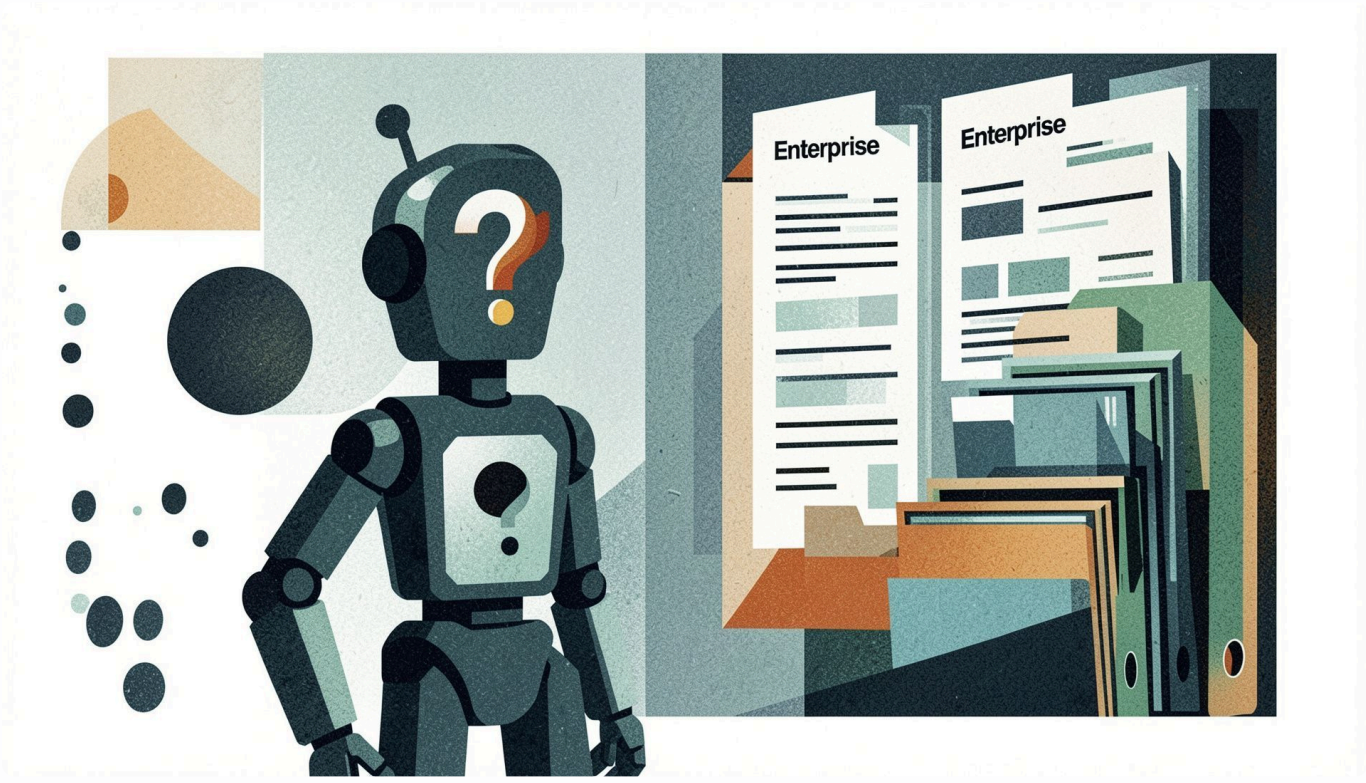
RAG — Giving LLMs Access to Enterprise Knowledge

AI@ZOOMERSKOOL

> *Can an LLM answer questions about our organization's private and constantly changing information?*

Made with GAMMA

Why Do We Need RAG?



Knowledge Cutoff

LLMs are trained up to a fixed date — they don't know your company's policies, recent decisions, or proprietary processes.

Enterprise Knowledge Changes

Policies update, procedures evolve, markets shift. The LLM's training data is stale the moment it ships.

Hallucination Risk

Without grounding, LLMs confidently generate plausible-sounding but incorrect answers — dangerous in banking.

The Solution

Give the LLM access to your **actual documents in real-time**. A loan officer asks about refinancing policy → RAG retrieves the current policy → cited, accurate answer.

Made with GAMMA

What Is RAG?

RAG = Retrieval-Augmented Generation



Retrieval

Find relevant documents or knowledge from your enterprise database

Augmented

Add that knowledge to the LLM's input — the prompt — as context

Generation

LLM generates an answer using both its training and the retrieved knowledge

Made with GAMMA

RAG Architecture: The Complete Pipeline

Setup Phase *(one-time)*

1. Documents
2. Split into Chunks
3. Convert to Embeddings
4. Store in Vector DB

Runtime Phase *(every query)*

1. User Question
2. Convert to Embedding → Search Vector DB
3. Retrieve Relevant Chunks
4. Send to LLM → Generate Answer

Made with GAMMA

Chunking, Embeddings, and Vector Search

Chunking

Split large documents into focused ~500-word pieces, each tagged with source and section. A 50-page policy becomes 100 precise, retrievable chunks.

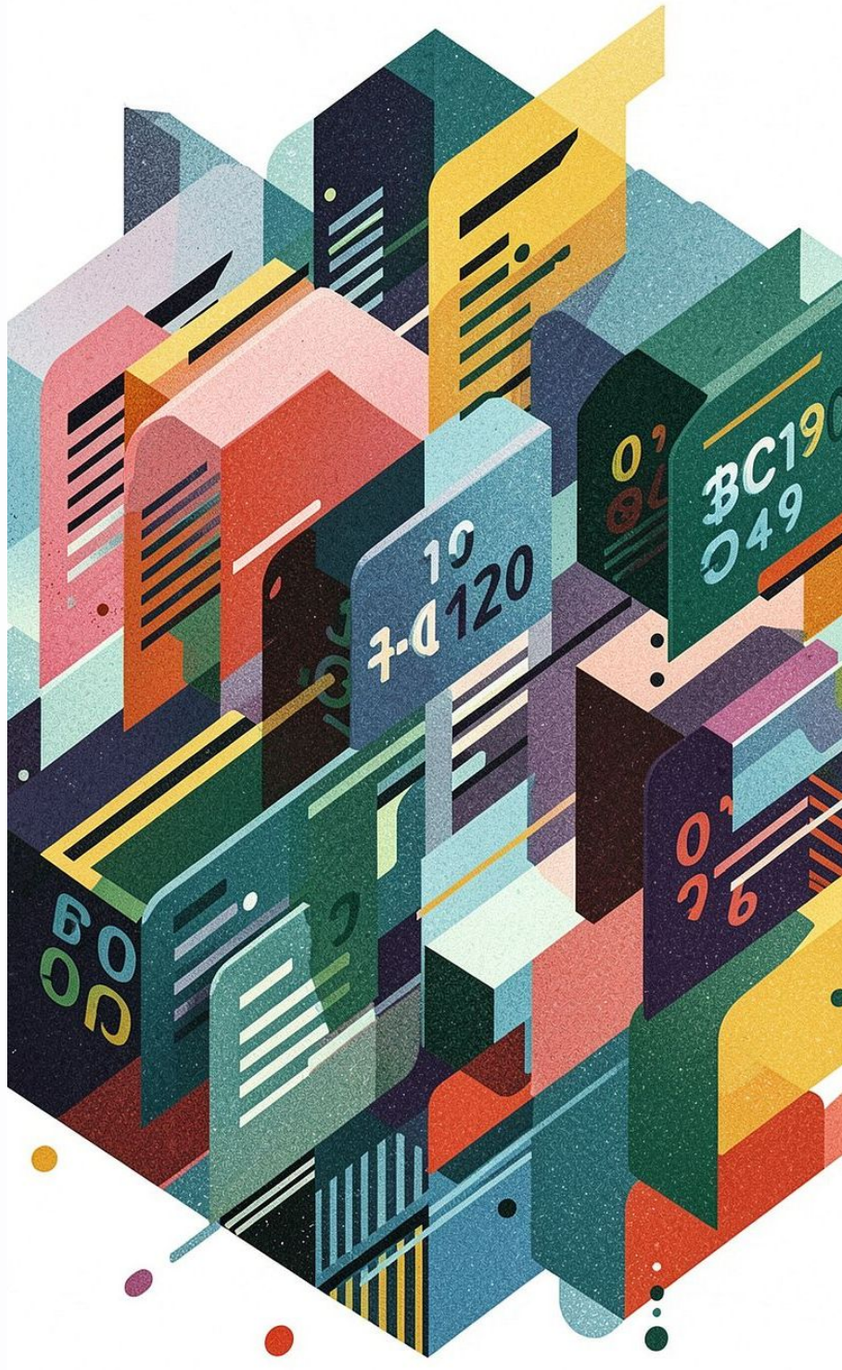
01
10

Embeddings

Convert text into a list of numbers — a "vector" — that encodes meaning. Similar ideas produce similar vectors. Computers compare numbers at massive scale.

Vector Search

Convert the user's question into a vector, then find the most similar chunk vectors. "What's our refinancing policy?" → returns refinancing chunks, not mortgage rates.



Made with GAMMA

What Happens When a User Asks a Question?



Scenario: Loan officer asks — "Can a customer with a 640 credit score refinance their mortgage?"

01

Query Understanding

System identifies intent: refinancing + credit score threshold

02

Retrieval

Question → vector → top 3–5 chunks from Refinancing Policy retrieved

03

Context Found

"Min score: 650. Exception: 620–649 may qualify with co-signer."

04

Augmented Prompt

System builds: [Policy excerpts] + [User question] → sent to LLM

05

Grounded Answer with Citation

"640 score does not meet the 650 standard, but a co-signer may qualify. See §3.2, Refinancing Policy (Jan 2025)."

Made with GAMMA

Traditional RAG: Strengths and Limitations

✓ Where Vector RAG Excels

Semantic Document Search

Finds content by meaning, not just keywords

FAQ & Knowledge Bases

Fast, accurate answers from large policy libraries

Policy Lookups

Low setup overhead, highly scalable

⚠️ Where It Struggles

Relationship-Heavy Questions

"What policies apply to Midwest customers with scores 620–650?" — requires connections across entities

Multi-Hop Reasoning

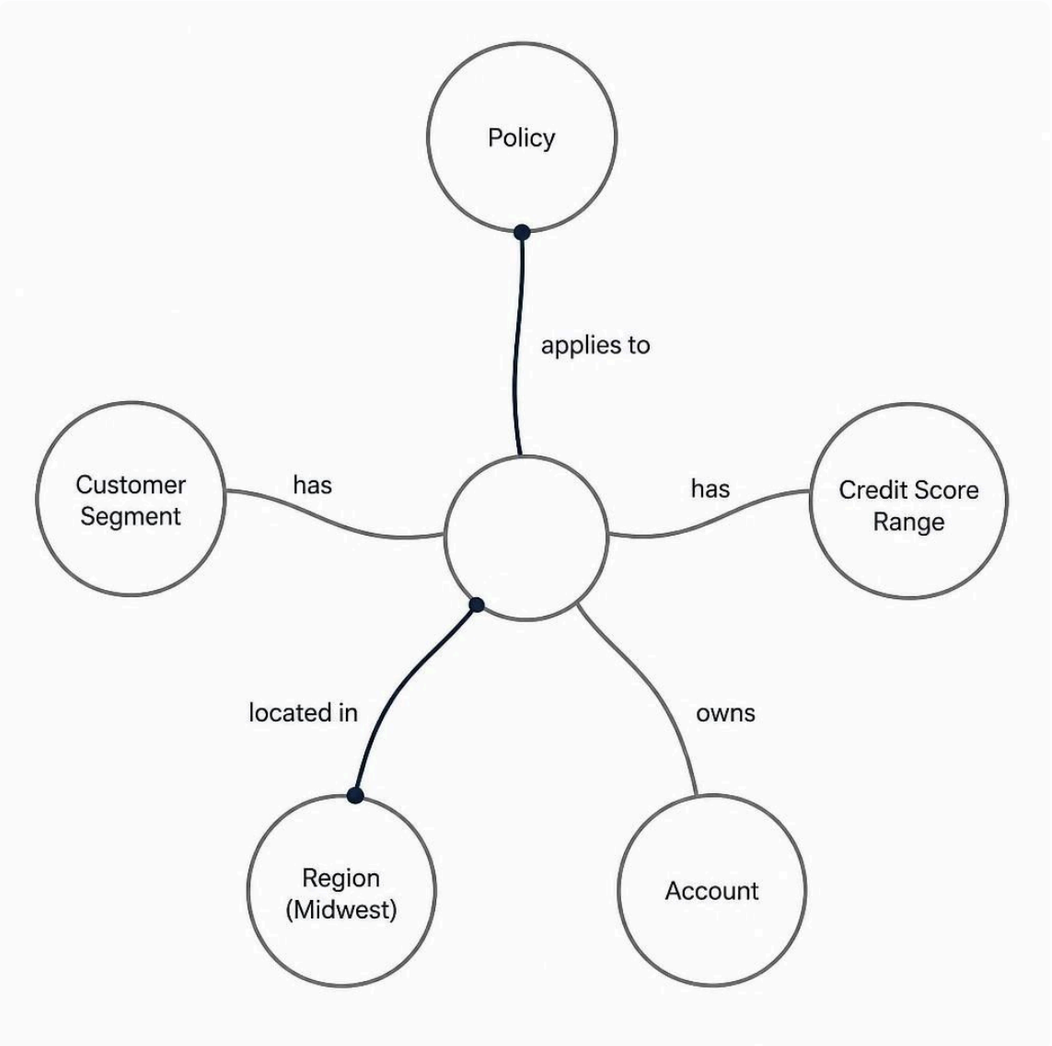
Chunks retrieved independently — system doesn't see how Policy A and Policy B interact

Keyword Mismatch

Policy says "co-signer," user says "guarantor" — system may miss relevant chunks

Made with GAMMA

GraphRAG and Knowledge Graphs



A Different Approach to Retrieval

Knowledge Graph: Entities as nodes, relationships as edges. *Policy → applies to → Customer Segment → located in → Region*

GraphRAG traverses these connections instead of — or alongside — vector similarity. Multi-step reasoning becomes natural.

Approach	Core Question
Vector RAG	How similar is this chunk to the query?
Graph RAG	What entities are connected to this entity?

Neither is universally better — they solve different problems. Choose based on your data structure.

Made with GAMMA

Enterprise RAG Architecture

Production RAG systems require more than just retrieval and generation. Guardrails are not optional.



Security & Access Control

Only retrieve documents the user is permitted to see — a junior analyst never surfaces executive data



PII Handling

Mask or redact sensitive data before ingestion. Comply with GDPR, CCPA, and internal data policies



Hallucination Prevention

If no relevant chunks are found, the system must say "I don't have information on this" — never guess



Citations & Traceability

Always include source chunk and metadata so users can verify answers and auditors can trace decisions



Evaluation & Monitoring

Measure retrieval precision and answer accuracy continuously — production RAG degrades without monitoring

Made with GAMMA

Key Takeaways & Interactive Quiz

Five Concepts to Remember

1. **RAG = Retrieval + Generation**

   Grounds LLM answers in actual enterprise data

2. **Embeddings = Meaning as Numbers**

   Enables fast semantic similarity search

3. **Vector DBs = Scalable Search**

   Find relevant chunks across millions of documents

4. **Knowledge Graphs = Relationships**

   Multi-step reasoning across connected entities

5. **Enterprise RAG = Guardrails**

   Security, PII, citations, and evaluation are non-negotiable

🗣️ Discussion

1

Why can't we just put all enterprise documents into the LLM's prompt?

Hint: context window limits, cost, relevance precision

2

What does an embedding actually represent?

Hint: a mathematical representation of semantic meaning

3

When is graph-based retrieval more useful than vector search?

Hint: relationship-heavy questions, multi-hop reasoning across entities

Made with GAMMA